Round 1 · Code Craft Challenge

60 dk · HackerRank CodePair · Go · kamera açık, ekran paylaşımı · **AI kullanımı yasak**

0-5 dk	5-50 dk · 45 dk kodlama	50-60 dk
Tanışma	Business problem, parça parça gelir. LeetCode değil, çalışan bir çözüm.	Sorular + tartışma

Prep kit'e göre neye bakıyorlar

- **Problem Solving:** belirsiz problemi nasıl ele alıyorsun? İlk seferde mükemmel değil, **iterasyonla** ilerleme.
- **Communication:** düşünceni anlatma, geri bildirim alıp yön değiştirme. Kod çalışmıyorsa bile iletişim.
- **Code Clarity:** temiz, düzenli, doğru sözdizimli kod. **Test ettin mi, debug ettin mi?**
- Puanlanan konular: maintainability, readability, **testability**, scalability, **failure handling**, **API design & development**, **OOP ve design pattern'ler**.

Recruiter notların (özet)

- Algoritma yok, **çalışan çözüm** bekleniyor. Hands-on yaklaşım.
- Business problem içine paketlenmiş: muhtemelen stok, satın alma, depo, fiyat gibi grocery & retail konuları.
- "Bazen Google'a izin veriliyor": pkg.go.dev bakmak için sor. **AI asistan yok.**
- Ekip: 5-8 kişi, cross-functional, cross-location. Go ağırlıklı, şirket geneli Kotlin/Python/Scala.

İpuçları (prep kit)

- **Hemen kod yazma.** Problemi anlamaya zaman ayır.
- Çok **interaktif** olacak: netleştirme soruları sormak puan kazandırır.
- Takılırsan yaklaşımını ve gerekçeni anlat → ipucu almanı sağlar.

Lojistik

□ Mülakatçı: bir Software Engineer + 1-2 gözlemci olabilir.
□ Tarih/saat: davetteki tarihi ve CET saatini **teyit et** (yeniden planlandıysa yeni davete bak).
□ Kamera açık, tüm ekranı paylaşmaya hazır ol. Copilot / AI eklentilerini **kapat**, gereksiz pencereleri kapat.
□ HackerRank'te Go seç; `package main`, stdlib. Bir kez "Run" deneyip çıktı formatını gör.
□ Kağıt + kalem: veri modeli ve soru listesi için.

Round 2 önizleme (geçersen, tek günde 4 × 1 saat)

AI builder	Kendi AI kurulumunla (Claude Code vb.) süreli bir görevi bitir. Nasıl yönlendirdiğine, doğruladığına bakıyorlar. Kod mükemmel değil, **tamam** olsun.
Debugging	Bir POC'yi oku, ne yaptığını anla, hataları düzelt, çalışır hale getir.
System design	Klasik whiteboard. (AI Company paketindeki kalıplar burada da işe yarar.)
Hiring manager	Engineering values, bireysel katkı, geçmişten örnekler, STAR.

Hedef: 45 dakikada **Part 1 + 2'yi temiz** bitirmek iyi sonuç; Part 3 çok iyi; Part 4 bonus. Puan; bitirmekten çok **nasıl düşündüğün, nasıl iletişim kurduğun ve kodun kalitesi** üzerinden geliyor. Elindeki 4 mock (inventory, purchase orders, pricing, stock events) bu formatın birebir simülasyonu. Bu paket onları **tamamlıyor**: Go hafıza kartları (s. 3-5), ısınma ve aggregator (s. 6-7), kağıt drill'leri (s. 8-11) ve cevapları (s. 12-13).

45 dakikanın akışı ve söylenecekler

Dakika	Adım	Ne yap	Ne söyle
0-4	Anla	Problemi kendi cümlelerinle özetle. 3-6 netleştirme sorusu. Varsayımları **yaz** (kodun en üstüne yorum olarak).	🗣 *Let me repeat it back to make sure I got it right… A few questions before I start.*
4-7	Tasarla	Tipler, ana struct, metod imzaları, hatalar. 30 sn'de veri modelini anlat.	🗣 *I'll model it like this: a Service struct that owns the state, typed IDs, sentinel errors. In-memory first.*
7-20	Part 1	Validasyon → kontrol → uygula. `main()`'de 3-4 senaryo, Run.	🗣 *I'm validating everything first so a failure never leaves half-applied state.*
20-32	Part 2	Yeni gereksinim. Önce mevcut tasarıma nasıl oturduğunu söyle, sonra kod.	🗣 *This fits nicely if I extract X into its own type…*
32-42	Part 3	Genişletilebilirlik (interface/strategy), zaman, eşzamanlılık.	🗣 *Rules will keep changing, so I'll put them behind a small interface.*
42-45	Toparla	Son Run, edge case'leri göster, kalanları söyle.	🗣 *With more time I'd add table-driven tests for… and persist this in Postgres.*
+10	Tartışma	Production'a nasıl taşınır? Eşzamanlılık, DB, Kafka event'leri, izleme, ölçek. (Sayfa 5'te cevap bankası.)	

Business problem'de evrensel netleştirme soruları

1. **Girdi validasyonu:** negatif/sıfır miktar, boş ID, bilinmeyen SKU → hata mı, yok saymak mı?
2. **Tekrar eden istek:** aynı ID ile tekrar gelirse? Aynı içerik → başarı, farklı içerik → conflict?
3. **Kısmi başarı:** ya hep ya hiç mi? Yoksa kısmi + rapor mu?
4. **Sıralama / eşitlik:** çıktı hangi sırada? Eşitlikte kural ne? (determinizm)
5. **Zaman:** süre dolması, timezone, "şimdi" kimden gelir? (Clock enjekte)
6. **Para:** hangi para birimi, yuvarlama kuralı? (integer cents)
7. **Durumlar:** hangi geçişler geçerli? İptal edilmiş bir şey tekrar açılır mı?
8. **Ölçek / eşzamanlılık:** tek process mi, birden çok goroutine çağırır mı?
9. **Hata türü:** çağıran hatayı nasıl ayırt edecek? (sentinel / typed error)

İletişim cümleleri

🗣 *I'll start with the simplest version that works and then iterate.*

🗣 *I'm making an assumption here: X. If that's wrong it's an easy change because…*

🗣 *Let me run it quickly before moving on.*

🗣 *That output isn't what I expected. Let me read it… I think the bug is in the ordering, because…*

🗣 *I'm choosing a map keyed by store and SKU because every operation looks up by both.*

🗣 *I'll return a copy here so callers can't mutate our internal state.*

🗣 *I'd normally write this as a table-driven test; given the time I'll put a few checks in main.*

🗣 *Good catch, thanks. Let me fix that.*

Takılırsan (sessiz kalma!)

1. Ne yapmaya çalıştığını söyle.
2. Basit bir örnekle elle izle (sayılarla).
3. Basitleştir: önce O(n) tarama, sonra optimize.
4. "Do you have a preference between A and B?" diye sormak serbest.

> Sözdizimi hatası olursa panik yok: derleyici mesajını sesli oku, düzelt. Bu da "debug ettin mi?" puanı.

Go tek dosya iskeleti (ezberle, 3 dakikada yaz)


```
package main

import (
  "errors"
  "fmt"
  "sort"
  "sync"
  "time"
)

// Assumptions (agreed with interviewer):
// - in-memory, single process; methods are safe for
concurrent use
// - quantities are ints > 0; money is int64 cents

// ---------- domain types ----------
type (
  SKU     string
  StoreID string
  OrderID string
)

type Status string

const (
  StatusOpen      Status = "OPEN"
  StatusDone      Status = "DONE"
  StatusCancelled Status = "CANCELLED"
)

// one place for valid transitions
var transitions = map[Status][]Status{
  StatusOpen: {StatusDone, StatusCancelled},
}

func canMove(from, to Status) bool {
  for _, s := range transitions[from] {
    if s == to {
      return true
    }
  }
  return false
}

// ---------- errors ----------
var (
  ErrInvalidArgument = errors.New("invalid argument")
  ErrNotFound        = errors.New("not found")
  ErrConflict        = errors.New("conflict")
  ErrInvalidState    = errors.New("invalid state")
)

type ShortageError struct {
  SKU                 SKU
  Requested, Available int
}

func (e *ShortageError) Error() string {
  return fmt.Sprintf("shortage %s: requested %d,
available %d", e.SKU, e.Requested, e.Available)
}

// ---------- time is injected ----------
type Clock interface{ Now() time.Time }
type realClock struct{}

func (realClock) Now() time.Time { return time.Now() }

type fakeClock struct{ t time.Time }

func (f *fakeClock) Now() time.Time          { return f.t
}
func (f *fakeClock) Advance(d time.Duration) { f.t =
f.t.Add(d) }
```

```
// ---------- service ----------
type Service struct {
  mu    sync.Mutex
  clock Clock
  stock map[StoreID]map[SKU]int
  // orders map[OrderID]*order ...
}

func NewService(c Clock) *Service {
  if c == nil {
    c = realClock{}
  }
  return &Service{clock: c, stock:
map[StoreID]map[SKU]int{}}
}

func (s *Service) Receive(store StoreID, sku SKU, qty
int) error {
  if store == "" || sku == "" || qty <= 0 {
    return fmt.Errorf("%w: store=%q sku=%q qty=%d",
      ErrInvalidArgument, store, sku, qty)
  }
  s.mu.Lock()
  defer s.mu.Unlock()
  if s.stock[store] == nil {
    s.stock[store] = map[SKU]int{}
  }
  s.stock[store][sku] += qty
  return nil
}

// deterministic output: sort keys
func (s *Service) Snapshot(store StoreID) []string {
  s.mu.Lock()
  defer s.mu.Unlock()
  skus := make([]SKU, 0, len(s.stock[store]))
  for k := range s.stock[store] {
    skus = append(skus, k)
  }
  sort.Slice(skus, func(i, j int) bool { return skus[i] <
skus[j] })
  out := make([]string, 0, len(skus))
  for _, k := range skus {
    out = append(out, fmt.Sprintf("%s=%d", k,
s.stock[store][k]))
  }
  return out
}

// ---------- demo + mini tests ----------
func check(name string, ok bool) {
  if ok {
    fmt.Println("PASS", name)
  } else {
    fmt.Println("FAIL", name)
  }
}

func main() {
  clk := &fakeClock{t: time.Date(2026, 11, 3,
    12, 0, 0, 0, time.UTC)}
  s := NewService(clk)

  err := s.Receive("hel-1", "milk", 0)
  check("zero qty rejected",
    errors.Is(err, ErrInvalidArgument))

  check("receive ok",
    s.Receive("hel-1", "milk", 5) == nil)
  fmt.Println(s.Snapshot("hel-1"))

  // typed error example:
  var se *ShortageError
  err = fmt.Errorf("reserve: %w",
    &ShortageError{"milk", 9, 5})
  check("errors.As works", errors.As(err, &se) &&
    se.Available == 5)
}
```

Neden bu iskelet? Mülakatçıya ilk 3 dakikada şunu gösterir: tipli ID'ler, tek yerde durum geçişleri, sentinel + typed error, enjekte edilen zaman, mutex, deterministik çıktı ve **kendi testini yazma alışkanlığı**. Hepsi rubrikteki "testability, failure handling, readability". **Ama hepsini baştan yazma:** önce map + küçük struct ile çalışan ilk dilim; `Clock`'u zaman gereksinimi gelince, mutex'i eşzamanlılık istenince, interface'i test veya genişleme gerektirince ekle. İlk 10 dakikayı iskelete harcama.

Go hafıza kartları (internet yokken tekrar et)

Hatalar

```
var ErrX = errors.New("x")                 // sentinel
return fmt.Errorf("%w: sku %s", ErrX, sku)  // wrap + context
errors.Is(err, ErrX)                        // zincirde ara
var se *ShortageError
errors.As(err, &se)                         // tipe çevir

// typed error'ı sentinel ile eşleştirmek:
func (e *ShortageError) Is(t error) bool { return t == ErrInsufficientStock }
// veya: func (e *ValidationError) Unwrap() error { return ErrRuleViolation }
// birden fazla: errors.Join(err1, err2)   (Go 1.20+)
```

Map'ler

```
m := map[SKU]int{}            // nil map'e yazma → panic!
v, ok := m[k]                 // comma-ok
delete(m, k)
// iç içe map: önce nil kontrolü
if s.stock[st] == nil { s.stock[st] = map[SKU]int{} }
// composite key alternatif:
type key struct{ Store StoreID; SKU SKU }
m2 := map[key]int{}; m2[key{"hel-1","milk"}]++
// iterasyon sırası RASTGELE → sırala
```

Slice'lar ve sıralama

```
out := make([]T, 0, len(in))
cp := append([]T(nil), s...)       // kopya (veya slices.Clone)
sort.Slice(xs, func(i, j int) bool {
    if xs[i].Expiry.Equal(xs[j].Expiry) { return xs[i].ID < xs[j].ID }
    return xs[i].Expiry.Before(xs[j].Expiry)
})
// Go 1.21+:
slices.SortFunc(xs, func(a, b Lot) int {
    if c := a.Expiry.Compare(b.Expiry); c != 0 { return c }
    return cmp.Compare(a.ID, b.ID)
})
slices.Contains(xs, v);  min(a, b);  max(a, b)
// for range içinde eleman KOPYA: pointer gerekiyorsa
for i := range xs { xs[i].Qty-- }
```

Zaman

```
d := 15 * time.Minute
deadline := now.Add(d);  now.After(deadline);
t2.Sub(t1) // Duration
y, w := t.ISOWeek()      // haftalık limitler
t.Truncate(24*time.Hour) // UTC gün başı (yerel saatte dikkat!)
loc, _ := time.LoadLocation("Europe/Helsinki");
t.In(loc)
d.Hours()  // float64
```

Para

```
type Cents int64
// %15 indirim, half-up yuvarlama (pozitif sayılar):
func applyPct(p Cents, pct int64) Cents {
    return Cents((int64(p)*(100-pct) + 50) / 100)
}
func (c Cents) String() string {
    return fmt.Sprintf("%d.%02d", c/100, c%100)
}
// ASLA float64 ile para hesaplama.
```

Interface / Strategy

```
type Rule interface {
    Name() string
    Check(ctx RuleContext) *Violation   // nil = OK
}
type MaxHours struct{ Max time.Duration }
func (r MaxHours) Name() string { return "max_hours" }
func (r MaxHours) Check(c RuleContext) *Violation { ... }

rules := []Rule{NoOverlap{}, MinRest{11*time.Hour}, MaxHours{40*time.Hour}}
// fonksiyon tipi de olur:
type PriceRule func(Line) (Cents, bool)
```

Eşzamanlılık

```
type S struct { mu sync.Mutex; ... }   // zero value kullanılabilir
s.mu.Lock(); defer s.mu.Unlock()
// TUZAK: kilitli public metottan başka kilitli public metodu çağırma
// → deadlock. İç yardımcılar kilitsiz: func (s *S) availableLocked(...)
sync.RWMutex: RLock() sadece okuma için
// struct'ı değer olarak kopyalama (mutex kopyalanır) → pointer receiver
```

Priority queue (container/heap)

```
type pq []*Task
func (q pq) Len() int            { return len(q) }
func (q pq) Less(i, j int) bool  { return q[i].Due.Before(q[j].Due) }
func (q pq) Swap(i, j int)       { q[i], q[j] = q[j], q[i] }
func (q *pq) Push(x any)         { *q = append(*q, x.(*Task)) }
func (q *pq) Pop() any {
    old := *q; n := len(old); t := old[n-1]
    *q = old[:n-1]; return t
}
heap.Init(&q); heap.Push(&q, t); t := heap.Pop(&q).(*Task)
```

Mülakatta önce **O(n) tarama** yaz, sonra "at scale I'd use a heap" de. Erken optimizasyon zaman yer.

fmt

```
%v değer  %+v alan adlarıyla  %q tırnaklı string  %T tip  %6.2f
strings.Builder; strings.Join(xs, ", ");
strconv.Itoa(n)
```

Tekrar eden kalıplar + Part 5 tartışma cevapları

9 refleks (her problemde)

1. Idempotency	Aynı ID + aynı içerik → önceki sonucu dön. Farklı içerik → `ErrConflict`. Sakla: `map[OrderID]record`.
2. Validate → check → apply	Önce tüm girdiyi doğrula, sonra **tüm** satırları kontrol et, en son değiştir. Hata = state değişmedi.
3. Durum makinesi	Geçişler tek map'te. Geçersiz → `ErrInvalidState` (+ from/to bilgisi).
4. Para = int cents	Yuvarlama kuralını söyle ve tek fonksiyonda tut.
5. Determinizm	Map sırası rastgele: çıktıyı sırala, eşitlikte ikinci anahtar (ID).
6. Clock enjekte	TTL/expiry'yi test edebilmek için. **Lazy expiry**: işlem anında kontrol et, arka plan goroutine'i gerekmez.
7. Typed error	Çağıran `errors.Is/As` ile ayırt eder. Tüm eksikleri listele (sıralı), ilkinde durma.
8. Kopya dön	İç slice/map'i dışarı verme.
9. Tek mutex, sonra konuş	"In production this lock becomes a DB transaction / row lock / partition."

Kodu "tamam" demeden önce 60 sn kontrol

- □ Sıfır / negatif / boş girdi denendi mi?
- □ Aynı istek iki kez → aynı sonuç?
- □ Hata sonrası state değişmedi mi? (göster!)
- □ Çıktı sıralı mı?
- □ Süre sınırı tam sınırda (== deadline) ne yapıyor?
- □ İsimler okunur mu? Uzun fonksiyonu böl.
- □ `main()` çıktısı mülakatçıya anlamlı mı?

Part 5: "How would this work in production?"

Persistence: 🗣️ "*State goes to Postgres. Reservations become rows; the check-and-apply runs in one transaction with `SELECT … FOR UPDATE` on the stock rows, or a conditional update: `UPDATE stock SET reserved = reserved + $q WHERE … AND on_hand - reserved >= $q`. Idempotency becomes a unique constraint on order_id.*"

Eşzamanlılık / ölçek: 🗣️ "*One mutex serializes everything. Next step is a lock per store, because stores are independent. At scale, partition by store: one Kafka partition or one worker owns a store, so there's a single writer and no cross-node locks.*"

Event'ler: 🗣️ "*Every state change emits an event, like ReservationCreated or StockReceived, through a transactional outbox so the DB write and the event can't diverge. Consumers are idempotent because delivery is at-least-once.*"

Sırasız / tekrarlı event: event_id ile dedupe, entity başına sequence/version; eski version'u yok say; boşluk varsa kısa buffer + timeout, sonra reconcile.

API: `POST /stores/{id}/reservations` + Idempotency-Key → 201; stok yok → 409 + eksik listesi; geçersiz → 400/422. gRPC'de `FAILED_PRECONDITION`, `ALREADY_EXISTS`.

Gözlemlenebilirlik: reservation başarı/red oranı, shortage oranı SKU bazlı, p99 latency, expired reservation sayısı, outbox lag. Log'da order_id, store_id.

Test: table-driven unit testler, fake clock, `go test -race`, eşzamanlı rezervasyon testi (100 goroutine, son ürün tek kişiye), DB ile entegrasyon testi (testcontainers).

Expiry production'da: lazy kontrol + periyodik sweeper job (ya da DB'de `expires_at` index + batch update). Saat kayması: tek kaynak (DB'nin `now()`).

Para / fiyat: fiyatı sipariş anında **snapshot**'la, sonradan değişen fiyat geçmiş siparişi değiştirmesin.

AI kullanımı (ilan istiyor; kendi pratiğine göre uyarla): 🗣️ "*I use AI assistants daily, for boilerplate, tests and exploring unfamiliar code, but I review and run everything and I own the result. Domain rules like rounding or state transitions I write myself, and I use AI to challenge the edge cases.*"

Isınma: tek mağaza stok rezervasyonu (kağıtta 15 dk)

Problem: Tek mağaza, in-memory servis. Bir sipariş için stok rezerve et ve rezervasyonu iptal et. İstekler tekrar gelebilir. Hata durumunda state değişmemeli. (Alıştırma kuralları; şirketten bildirilmiş bir soru değil.)

Sözleşme

- Bir order ID = bir SKU + pozitif tam sayı miktar.
- Aynı ID + aynı girdi → mevcut rezervasyonu dön. Aynı ID + farklı girdi → conflict.
- İptal stoğu **bir kez** geri verir; tekrar iptal etkisizdir.
- İptal edilmiş bir rezervasyonun tekrarı, iptal kaydını döner; yeniden rezerve **etmez**.
- Bilinmeyen iptal hata; olmayan SKU = 0 stok. Tek process, sıralı çağrılar, süre dolması yok.

Elle izle (önce kendin doldur)

İşlem	Beklenen	milk
Başlangıç milk = 5	Rezervasyon yok	5
Reserve A, milk, 3	A RESERVED	2
Tekrar A, milk, 3	Aynı rezervasyon	2
Reserve A, milk, 4	Conflict, değişiklik yok	2
Reserve B, milk, 3	Yetersiz stok, değişiklik yok	2
Cancel A; tekrar cancel A	A CANCELLED, bir kez iade	5
Tekrar A, milk, 3	CANCELLED kaydı döner	5

Kodu çevirmeden önce yaz

Hangi veriyi saklıyorsun? Stok hangi satırda değişebilir? Stoğa bakmadan **önce** hangi kontrol yapılmalı? Çağıranın iç kaydı değiştirmesini nasıl engellersin?

> **Kritik sıra:** validate → **mevcut siparişe bak** → stok kontrolü → değiştir. Stok kontrolünü replay kontrolünden önce yaparsan, ilk istek stoğu zaten düşürdüğü için aynı isteğin tekrarı "yetersiz stok" ile başarısız olur.

Go cevabı (derlendi, `go test -race` geçti)

```
package main

import "errors"

type Reservation struct {
  SKU   string
  Qty   int
  State string
}
type Store struct {
  stock  map[string]int
  orders map[string]Reservation
}

func NewStore(stock map[string]int) *Store {
  copyStock := make(map[string]int, len(stock))
  for sku, qty := range stock {
    copyStock[sku] = qty
  }
  return &Store{copyStock, map[string]Reservation{}}
}
func (s *Store) Reserve(id, sku string, qty int)
(Reservation, error) {
  if id == "" || sku == "" || qty <= 0 {
    return Reservation{}, errors.New("invalid input")
  }
  if old, ok := s.orders[id]; ok {
    if old.SKU != sku || old.Qty != qty {
      return Reservation{}, errors.New("conflicting
replay")
    }
    return old, nil
  }
  if s.stock[sku] < qty {
    return Reservation{}, errors.New("insufficient stock")
  }
  row := Reservation{sku, qty, "RESERVED"}
  s.stock[sku] -= qty
  s.orders[id] = row
  return row, nil
}
func (s *Store) Cancel(id string) (Reservation, error) {
  old, ok := s.orders[id]
  if !ok {
    return Reservation{}, errors.New("unknown order")
  }
  if old.State == "CANCELLED" {
    return old, nil
  }
  s.stock[old.SKU] += old.Qty
  old.State = "CANCELLED"
  s.orders[id] = old
  return old, nil
}
```

Dönüşler değer (struct kopyası), çağıran iç kaydı değiştiremez. Bu çekirdek sıralı çağrı varsayar; eşzamanlı çağrıda **tüm** Reserve/Cancel işlemini kilitle. Birden çok replika → DB transaction veya başka paylaşılan atomik otorite.

🗣️ *The main guarantee is that a repeated request can't change stock twice. I check the existing order before availability, and I keep each state transition together.*

Aggregator tekrarı: daha önce çalıştığın Go problemi

Sık sorulan bir pratik soru: Consumer zorunlu, Payment ve Address birbirinden bağımsız ve opsiyonel. Sonuçları ayrı değişkenlerde tutup WaitGroup bitince cevabı tek goroutine'de kurmuştun. Takip sorusu: **Payment'ın 500 ms bütçesi var.** (Bu bir mülakat alıştırması, production kodu değil.)

```
func (s BootstrapService) Get(ctx context.Context,
  id string) (BootstrapResponse, error) {
  if strings.TrimSpace(id) == "" {
    return BootstrapResponse{}, errors.New("invalid ID")
  }
  consumer, err := s.Consumer.GetConsumer(ctx, id)
  if ctx.Err() != nil {
    return BootstrapResponse{}, ctx.Err()
  }
  if err != nil {
    return BootstrapResponse{}, err
  }
  var wg sync.WaitGroup
  var payment Payment
  var address Address
  var paymentErr, addressErr error
  wg.Add(2)

  go func() {
    defer wg.Done()
    paymentCtx, cancel := context.WithTimeout(
      ctx, 500*time.Millisecond,
    )
    defer cancel()
    payment, paymentErr = s.Payment.GetPayment(
      paymentCtx, consumer.ID,
    )
  }()
  go func() {
    defer wg.Done()
    address, addressErr = s.Address.GetAddress(
      ctx, consumer.ID,
    )
  }()
  wg.Wait()

  if err := ctx.Err(); err != nil {
    return BootstrapResponse{}, err
  }
  // One goroutine builds the response from these local
results.
  // Treat an optional error as partial data, not total
failure.
  response := BootstrapResponse{Consumer: &consumer}
  if paymentErr == nil {
    response.Payment = &payment
  }
  if addressErr == nil {
    response.Address = &address
  }
  response.Partial = paymentErr != nil || addressErr != nil
  return response, nil
}
```

Sınırı tam olarak anlat

🗣️ *Payment gets its own child context with a 500 ms timeout. Address keeps the parent context. A Payment timeout becomes a partial response, not a failure. This relies on the Payment client respecting cancellation: a context doesn't kill a goroutine.*

- WaitGroup tamamlanmayı bekler; context'e uymayan bir client yine bloklayabilir. Transport ve client deadline'a uymalı.
- Parent iptali ile opsiyonel Payment timeout'u farklı şeyler: parent iptal → hata dön; Payment timeout → kısmi cevap.
- Paylaşılan state yok: her goroutine kendi değişkenine yazar, cevap `wg.Wait()`'ten sonra kurulur → race yok.

Kısa retry cevabı

🗣️ *I retry transient failures only when repeating the call is safe. I cap total attempts, back off with jitter, and stay inside the dependency's time budget. If most calls are failing, I'd rather shed load with concurrency limits and a circuit breaker. And I keep the retry policy in one layer, so retries don't multiply.*

Gösterilecek testler

- ☐ Consumer hatası → opsiyonel çağrılar yapılmaz.
- ☐ Bir opsiyonel hata, diğer sonucu korur (`Partial = true`).
- ☐ Parent iptali yayılır.
- ☐ Payment timeout'u Address'i iptal etmez.
- ☐ `ctx.Done()`'u bekleyen fake client ile timeout testi; `go test -race`.

Hatırlatma: önce doğru ve race-safe çözüm; context, timeout ve retry sonra, sorulunca. Erken soyutlama yok.

Kağıt drill 1 · Son kullanma tarihli stok (FEFO)

Nasıl çalışılır: 4 dk soru listesi yaz → tipleri ve imzaları kağıda yaz → ana fonksiyonun kodunu elle yaz → edge case'leri say → sayfa 12'deki cevap anahtarıyla karşılaştır.

Context: Şirketin dark store'larında taze ürünler (süt, salata, et) **parti (lot)** halinde gelir ve her partinin son kullanma tarihi farklıdır. İsraf (waste) büyük bir maliyet. Toplayıcılar (picker) her zaman **son kullanma tarihi en yakın** partiden almalı (FEFO: First Expired, First Out). Bu çekirdek bileşeni yaz. In-memory, tek mağaza ile başla.

Part 1 (~12 dk) · Partiler

- `ReceiveLot(lot Lot) error`: yeni parti geldi. `Lot{ID, SKU, Qty, Expiry}`
- `OnHand(sku SKU) int`: satılabilir (süresi dolmamış) toplam adet.

Part 2 (~12 dk) · FEFO toplama

- `Pick(order OrderID, sku SKU, qty int) ([]Allocation, error)`: hangi partiden kaç adet alınacağını döner, stoktan düşer. `Allocation{LotID, Qty}`

Part 3 (~10 dk) · İsraf ve indirim

- `WriteOffExpired() []WriteOff`: süresi dolanları stoktan çıkar, rapor dön.
- `MarkdownSuggestions() []Markdown`: süresi yaklaşan partiler için indirim önerisi. Kural: 1 gün kala %50, 2 gün kala %30, 3 gün kala %10. **Kural ülkeye göre değişebilir.**

Part 4 (bonus) · Çoklu mağaza ve event'ler

- Birden fazla mağaza; her stok değişikliği bir event üretsin (`LotReceived`, `Picked`, `WrittenOff`). Eşzamanlı güvenli.

Senin soru listen

Senin veri modelin

Pick() kodunu buraya elle yaz

Kağıt drill 2 · Toplama görevi atama (warehouse execution)

Context: Bir dark store'a gelen her sipariş için bir **pick task** oluşur. Siparişin müşteriye söz verilen bir teslim saati (`PromisedAt`) var. Picker'lar el terminalinden "sıradaki görev" ister. Amaç: geç kalan sipariş sayısını azaltmak ve hiçbir görevin "kayıp" olmaması. Görev atama servisini yaz.

Part 1 (~12 dk) · Görev oluştur ve ata

- `CreateTask(order OrderID, promisedAt time.Time) (TaskID, error)`
- `NextTask(picker PickerID) (Task, error)` : en acil görevi verir.
- `Complete(task TaskID, picker PickerID) error`

Part 2 (~12 dk) · Zaman aşımı ve iptal

- Atanan görev 15 dk içinde tamamlanmazsa tekrar kuyruğa döner (picker telefonu düşürdü, mola verdi…).
- `Cancel(order OrderID) error`: müşteri iptal etti. Atanmış görev de iptal edilebilir.
- `Release(task, picker) error`: picker görevi bırakır.

Part 3 (~10 dk) · Öncelik politikası

- Express siparişler (üyelik?) önce. Bazı mağazalar "en eski oluşturulan önce" ister. **Politika mağaza başına değiştirilebilir olmalı.**
- `LateTasks(now) []Task`: söz verilen saati geçmiş ve tamamlanmamış görevler (sıralı).

Part 4 (bonus) · Batch picking

- Bir picker aynı anda en fazla 3 görev alabilir, aynı bölgedeki (zone) görevler birlikte verilir.

Durum makinesini çiz

NextTask() kodunu elle yaz

Senin soru listen

Kağıt drill 3 · Vardiya planlama kuralları (labor scheduling)

Context: Grocery & Retail ekibi mağaza personeli için vardiya planlıyor. Bir planlamacı bir çalışanı bir vardiyaya atarken sistem iş kanununa ve operasyon kurallarına uymayan atamaları **engellemeli** ve **nedenini açıkça** söylemeli. Finlandiya ve İsveç'te kurallar farklı. Atama servisini yaz. (İlanda "labor scheduling" geçiyor.)

Part 1 (~12 dk) · Temel atama

- `AddWorker(w Worker) error` · `Worker{ID, Name, Roles []Role}` (picker, receiver, shift_lead)
- `AddShift(s Shift) error` · `Shift{ID, Store, Role, Start, End, Capacity}`
- `Assign(worker WorkerID, shift ShiftID) error`: kurallar: rol uyumu, vardiya dolu değil, çakışan vardiya yok.

Part 2 (~12 dk) · İş kanunu kuralları

- İki vardiya arasında en az 11 saat dinlenme.
- Haftada (ISO hafta) en fazla 40 saat.
- Atama reddedilirsa **tüm** ihlaller (sadece ilki değil) dönsün, sıralı ve okunur.

Part 3 (~10 dk) · Ülkeye göre kural seti

- Finlandiya: haftalık max 40 saat. İsveç: max 40 saat **ve** 24 saatte en fazla 13 saat çalışma. Yeni ülke eklemek kolay olmalı.
- `Unassign(worker, shift) error`

Part 4 (bonus) · Öneri

- `Suggest(shift ShiftID) []WorkerID`: bu vardiyaya kurallara uyarak atanabilecek çalışanlar; o hafta en az çalışan önce, eşitlikte ID.

Senin soru listen

Rule interface'ini tasarla

MinRest kuralının Check() kodu

İki kısa drill: sıralı stok event'leri ve fiyat hesabı

Drill A · Stok delta'larını sırayla uygula

Event: store, SKU, event ID, sequence, delta. Sequence her store/SKU için birer artar. Çağrılar sıralı. Aynı event ID iki kez uygulanmaz. Eksik sequence gelene kadar gelecek event'leri bekletin (buffer). Geçersiz event state'i kısmen değiştirmez.

```
Başlangıç: quantity=10, last_sequence=0
E1: id=a, sequence=1, delta=−3
E3: id=c, sequence=3, delta=+4
E1 tekrar
E2: id=b, sequence=2, delta=−2
E2 tekrar
Her çağrıdan sonra: quantity / last_sequence /
buffer
```

Aynı sequence farklı bir event ile gelirse? Boşluk hiç kapanmazsa? Elindekileri sıralayıp uygulamak doğru olur mu? Neden?

Drill B · Deterministik fiyat

Para integer cents. Sıra: miktar indirimi → tek kupon → sıfır tabanı. Yüzde indirimi tam cent'e **aşağı** yuvarla, sonra çıkar. Negatif miktar ve fiyatı reddet. (Alıştırma kuralı.)

```
Birim fiyat: 199 cent, miktar: 3
Miktar indirimi: miktar >= 3 ise %10
Kupon: indirimli ara toplamdan 100 cent

Ara toplam, indirim, son toplam?
Yuvarlama sınırını nasıl test edersin?
Kuponlar birleşemezse ne değişir?
```

Sırf ilanda "design patterns" yazıyor diye strategy hiyerarşisi kurma. Tek fonksiyon yeterli olabilir; birden çok kural ayrı davranış ve test gerektirince policy interface'ine çıkar.

Cevaplar (sayfayı çevir, önce çözi)

Girdi	quantity	last_seq	buffer
E1	7	1	boş
E3	7	1	3
E1 tekrar	7	1	3
E2	9	3	boş (önce 2, sonra 3)
E2 tekrar	9	3	boş

Delta önceki durumu bağlı: 2'yi atlayıp 3'ü uygulamak yanlış olabilir. Boşluğu boyut ve süre limitiyle bekle; kapanmazsa o anahtarı durdur, yetkili bir snapshot'tan uzlaştır. Aynı event ID/sequence'in çelişkili tekrarını normal duplicate sayma, hata olarak yakala. Production'da dedupe ve checkpoint state'i restart'tan sağ çıkmalı.

Drill B: 199 × 3 = **597**. %10 = 59,7 → aşağı **59**. İndirimli: **538**. Kupon sonrası: **438 cent**.

Testler: miktar 2 → indirim yok; miktar 0 → 0; kupon ara toplamdan büyük → 0; geçersiz girdi state'i değiştirmez; yuvarlama sınırı (ör. 10 cent × 3 = 30 → indirim tam 3; 11 cent × 3 = 33 → %10 = 3,3 → 3).

Production takipleri: birden çok replika → mutex yetmez, atomik update/transaction. Rezerve + yayınla → stok ve outbox aynı commit'te, duplicate teslimatı tolere et. Süre dolması → clock enjekte et, sınırı tanımla, stoğu korunan bir geçişle bir kez iade et.

Cevap anahtarı · Drill 1 (FEFO) ve Drill 2 (Pick tasks)

Önce kendin çöz, sonra bak. Tek doğru çözüm yok; önemli olan kararları söyleyebilmek.

Drill 1 · Sorman gerekenler (+ makul cevap)

- Aynı lot ID tekrar gelirse? → aynı içerik: no-op; farklı: `ErrConflict`.
- Süresi bugün dolan ürün satılabilir mi? → `Expiry` anında satılamaz (`!now.Before(expiry)`). Sınırı netleştir!
- Geçmiş tarihli lot gelirse? → `ErrInvalidArgument`.
- Yeterli stok yoksa kısmi toplama mı? → ya hep ya hiç, `*ShortageError`.
- Aynı expiry'de iki lot? → önce erken gelen / küçük ID (determinizm).
- Aynı order tekrar Pick → aynı allocation'ları dön (idempotent).

Drill 1 · Veri modeli

```
type Lot struct { ID LotID; SKU SKU; Qty int; Expiry
time.Time }
type Store struct {
  mu    sync.Mutex
  clock Clock
  lots  map[SKU][]*Lot          // FEFO sıralı
tutulur
  seen  map[LotID]Lot           // idempotency
(orijinal)
  picks map[OrderID]pickRecord  // idempotency
}
```

Drill 1 · Pick

```
func (s *Store) Pick(o OrderID, sku SKU, qty int)
([]Allocation, error) {
  if o == "" || qty <= 0 {
    return nil, fmt.Errorf("%w: order=%q qty=%d",
ErrInvalidArgument, o, qty)
  }
  s.mu.Lock(); defer s.mu.Unlock()
  if p, ok := s.picks[o]; ok {                    //
idempotent replay
    if p.sku != sku || p.qty != qty { return nil,
ErrConflict }
    return append([]Allocation(nil), p.allocs...), nil
  }
  now := s.clock.Now()
  avail := 0                                      //
1) check
  for _, l := range s.lots[sku] {
    if now.Before(l.Expiry) { avail += l.Qty }
  }
  if avail < qty {
    return nil, &ShortageError{SKU: sku, Requested: qty,
Available: avail}
  }
  var out []Allocation                            //
2) apply (FEFO)
  left := qty
  for _, l := range s.lots[sku] {
    if left == 0 { break }
    if !now.Before(l.Expiry) || l.Qty == 0 { continue }
    take := min(l.Qty, left)
    l.Qty -= take; left -= take
    out = append(out, Allocation{LotID: l.ID, Qty:
take})
  }
  s.picks[o] = pickRecord{sku, qty, out}
  return append([]Allocation(nil), out...), nil
}
```

Markdown (Part 3): `type MarkdownPolicy interface{ Discount(daysLeft int) (pct int, ok bool) }`; ülke başına politika (Strategy). Gün hesabını **mağazanın yerel saatinde** yap (timezone sorusu = artı puan).

Drill 2 · Sorman gerekenler

- Picker'ın zaten görevi varken NextTask? → aynı görevi dön (idempotent, el terminali tekrar dener).
- Kuyruk boşsa? → `ErrNoTask` (hata mı, boş sonuç mu? sor).
- Başka picker'ın görevini tamamlama? → `ErrNotOwner`.
- Zaman aşımına uğramış görevi eski picker tamamlarsa? → tekrar atanmadıysa kabul et (pragmatik) ya da reddet: **sor**.
- Eşit PromisedAt → oluşturulma sırası (seq).

Drill 2 · Durum makinesi

```
QUEUED ──NextTask──▶ ASSIGNED ──Complete──▶ DONE
   ▲                    │
   └──timeout/Release───┘
QUEUED | ASSIGNED ──Cancel──▶ CANCELLED   (DONE'dan
çıkış yok)
```

Drill 2 · NextTask (lazy timeout + O(n) seçim)

```
func (s *Service) NextTask(p PickerID) (Task, error) {
  if p == "" { return Task{}, ErrInvalidArgument }
  s.mu.Lock(); defer s.mu.Unlock()
  now := s.clock.Now()
  s.expireLocked(now)                     // ASSIGNED &&
now >= assignedAt+15m → QUEUED

  if id, ok := s.byPicker[p]; ok {        // already has
one: return it
    return *s.tasks[id], nil
  }
  var best *Task
  for _, t := range s.tasks {
    if t.Status != StatusQueued { continue }
    if best == nil || s.policy.Less(t, best) { best = t
}
  }
  if best == nil { return Task{}, ErrNoTask }
  best.Status, best.Picker, best.AssignedAt =
StatusAssigned, p, now
  s.byPicker[p] = best.ID
  return *best, nil                       // copy, not
pointer
}

type Policy interface{ Less(a, b *Task) bool }
type EarliestPromise struct{}
func (EarliestPromise) Less(a, b *Task) bool {
  if !a.PromisedAt.Equal(b.PromisedAt) { return
a.PromisedAt.Before(b.PromisedAt) }
  return a.Seq < b.Seq
}
type ExpressFirst struct{ Next Policy }   // decorator /
chain
func (e ExpressFirst) Less(a, b *Task) bool {
  if a.Express != b.Express { return a.Express }
  return e.Next.Less(a, b)
}
```

Söyle: "The scan is O(n); with thousands of queued tasks per store I'd use a heap, but the policy interface stays the same." · expireLocked'da `byPicker`'dan da sil (unutulan klasik bug!).

Cevap anahtarı · Drill 3 (Vardiya kuralları)

Sorman gerekenler

- Vardiya gece yarısını geçebilir mi? (22:00-06:00) → evet; haftalık saatte hangi haftaya sayılır? → başladığı hafta (basit kural, söyle).
- Zamanlar hangi timezone'da? → mağazanın yerel saati; ISO hafta yerel saate göre. DST günü vardiya 7 ya da 9 saat sürebilir: süreyi `End.Sub(Start)` ile hesapla, saat farkıyla değil.
- Uç uca iki vardiya (biri 14:00 biter, diğeri 14:00 başlar) çakışma mı? → hayır (yarı açık aralık [start, end)), ama dinlenme kuralına takılır.
- Aynı atama tekrar → idempotent no-op.
- Kural ihlalinde "override" (yönetici onayı) var mı? → kapsam dışı, sonra.

Tipler

```
type Violation struct { Rule, Detail string }

type ValidationError struct{ Violations []Violation }
func (e *ValidationError) Error() string {
  parts := make([]string, len(e.Violations))
  for i, v := range e.Violations { parts[i] = v.Rule + ": "
+ v.Detail }
  return "assignment rejected: " + strings.Join(parts, ";
")
}
func (e *ValidationError) Unwrap() error { return
ErrRuleViolation }

type RuleContext struct {
  Worker   Worker
  Shift    Shift
  Existing []Shift          // worker's current shifts
(copy)
}
type Rule interface {
  Name() string
  Check(c RuleContext) *Violation
}
// country → rules  (Part 3: add a country = add a line)
var rulesByCountry = map[string][]Rule{
  "FI": {RoleMatch{}, NoOverlap{}, MinRest{11 * time.Hour},
         MaxWeekly{40 * time.Hour}},
  "SE": {RoleMatch{}, NoOverlap{}, MinRest{11 * time.Hour},
         MaxWeekly{40 * time.Hour}, MaxInWindow{13 *
time.Hour, 24 * time.Hour}},
}
```

Assign

```
func (s *Service) Assign(wid WorkerID, sid ShiftID) error {
  s.mu.Lock(); defer s.mu.Unlock()
  w, ok := s.workers[wid]
  if !ok { return fmt.Errorf("%w: worker %s", ErrNotFound,
wid) }
  sh, ok := s.shifts[sid]
  if !ok { return fmt.Errorf("%w: shift %s", ErrNotFound,
sid) }
  if s.assigned[sid][wid] { return nil }            //
idempotent
  if len(s.assigned[sid]) >= sh.Capacity {
    return fmt.Errorf("%w: shift %s is full",
ErrInvalidState, sid)
  }
  ctx := RuleContext{w, sh, s.shiftsOfLocked(wid)}
  var vs []Violation
  for _, r := range rulesByCountry[s.countryOf(sh.Store)] {
    if v := r.Check(ctx); v != nil { vs = append(vs, *v) }
  }
  if len(vs) > 0 {
    sort.Slice(vs, func(i, j int) bool { return vs[i].Rule
< vs[j].Rule })
    return &ValidationError{Violations: vs}
  }
  if s.assigned[sid] == nil { s.assigned[sid] =
map[WorkerID]bool{} }
  s.assigned[sid][wid] = true
  return nil
}
```

MinRest ve MaxWeekly

```
type MinRest struct{ Min time.Duration }
func (MinRest) Name() string { return "min_rest" }
func (r MinRest) Check(c RuleContext) *Violation {
  for _, e := range c.Existing {
    var gap time.Duration
    switch {
    case !e.End.After(c.Shift.Start): gap =
c.Shift.Start.Sub(e.End) // e before
    case !c.Shift.End.After(e.Start): gap =
e.Start.Sub(c.Shift.End) // e after
    default: continue                          // overlap:
NoOverlap reports it
    }
    if gap < r.Min {
      return &Violation{"min_rest", fmt.Sprintf("only %v
rest around shift %s", gap, e.ID)}
    }
  }
  return nil
}

type MaxWeekly struct{ Max time.Duration }
func (MaxWeekly) Name() string { return "max_weekly_hours"
}
func (r MaxWeekly) Check(c RuleContext) *Violation {
  y, w := c.Shift.Start.ISOWeek()
  total := c.Shift.End.Sub(c.Shift.Start)
  for _, e := range c.Existing {
    if ey, ew := e.Start.ISOWeek(); ey == y && ew == w {
      total += e.End.Sub(e.Start)
    }
  }
  if total > r.Max {
    return &Violation{"max_weekly_hours", fmt.Sprintf("%v >
%v", total, r.Max)}
  }
  return nil
}
```

Suggest (Part 4): her aday için aynı kuralları çalıştır (Assign'ı yeniden kullan: `validateLocked` diye ayır, kopyala-yapıştır yapma), geçenleri haftalık saate göre sırala, eşitlikte ID.

Perakende sözlüğü + son tekrar

Business problem'i hızlı anlamak için alan terimleri. Mülakatçı bunlardan birini kullanırsa anlamını bilmek güven verir; bilmiyorsan **sormak da** iyi: "Just to make sure, by case pack you mean…?"

Terim	Anlamı · koddaki karşılığı
Dark store	Müşteriye kapalı mini depo; sadece online siparişler. Mağaza = `StoreID`.
SKU	Stock Keeping Unit: tekil ürün varyantı (1L süt Valio ≠ 1,5L).
On-hand / Available	Fiziksel stok / satılabilir = on-hand − reserved − (hasarlı, süresi dolmuş).
Reservation / allocation	Checkout'ta stoğu siparişe ayırmak. TTL ile serbest kalır.
Replenishment	Stok yenileme. **Reorder point** altına düşünce **target level**'a tamamla.
Safety stock	Talep belirsizliğine karşı tampon stok.
Lead time	Siparişten teslime geçen süre. Sipariş miktarı = tahmini talep × (lead time + review period) + safety − (on-hand + in-transit).
Case pack / MOQ	Tedarikçi koli adedi (12'li) / minimum sipariş miktarı → yukarı yuvarla: `(q + pack − 1) / pack * pack`.
PO (Purchase Order)	Tedarikçiye verilen sipariş. Durumlar: draft → submitted → confirmed → (partially) received → closed.
ASN	Advance Shipping Notice: tedarikçinin "şunları gönderdim" bildirimi.
GRN / Goods receipt	Depoda mal kabul kaydı (sayılan gerçek miktar).
Three-way match	PO ↔ mal kabul ↔ fatura eşleştirme, tolerans dahilinde → ödeme onayı.
FEFO / FIFO	İlk son kullanma tarihi dolan önce çıkar / ilk giren önce çıkar.
Shrink / waste	Kayıp: çalınma, hasar, son kullanma tarihi geçmesi. Write-off.
Markdown	Süresi yaklaşan ürüne indirim.
Planogram / slotting	Ürünün raftaki yeri; toplama rotasını etkiler (zone, aisle).
Pick / pack / dispatch	Raftan topla → paketle → kuryeye teslim. Her biri bir durum.
Cycle count	Stok sayımı; sistem ile gerçek arasındaki fark → **adjustment** event'i.
Substitution	Ürün yoksa müşteri onayıyla benzeri verilir.
Forecast	SKU × mağaza × gün talep tahmini; replenishment'ın girdisi.
Price zone / national price	Bölge / ülke fiyatı; mağaza override > zone > national önceliği.

Mülakattan 10 dk önce oku

1. Kod yazmadan önce: özetle, sor, varsayımları yaz.
2. İskelet: tipler, hatalar, Clock, Service + mutex, main + check.
3. Validate → check → apply. Hata = state aynı.
4. Idempotency, durum makinesi, sıralı çıktı, cents.
5. Her part'tan sonra **Run**. Sesli düşün.
6. Sonda: Postgres, per-store lock/partition, outbox + Kafka, metrikler, -race.

Onlara sorabileceğin sorular

1. What does a typical week look like for an engineer in this team?
2. How do you split work between Go services and the forecasting/data platform?
3. What was the hardest part of replacing the third-party forecasting solution?
4. How does on-call work, and how often do you get paged?
5. How do teams in Helsinki and Stockholm collaborate day to day?